

Week-8 Assignment: E-Commerce Order Analytics System

End-to-End Order Data Processing and Analytics using Python and SQL

Dataset: E-Commerce Orders Dataset (Customers, Products, Orders, Order Items)

Author: Snehal A. Bhosale

College: Sanjivani College of Engineering, Kopergaon – 423603

E-mail: snehalbhosale1807@gmail.com

CEI ID: CT_CSI_DE_1177

Technology Used

- Python
- Pandas
- MySQL / MySQL Workbench
- SQLite
- Jupyter Notebook
- SQL (Joins, CTEs, Window Functions)
- CSV File Format

1. Objective

The objective of this assignment is to design and implement an end-to-end e-commerce order analytics system. The assignment focuses on cleaning and validating raw order data, loading it into a relational database, writing SQL-based analytics using joins, aggregations, window functions, and CTEs, and building a Python command-line tool to generate business reports.

2. Introduction

E-commerce platforms generate large volumes of data through customer registrations, product listings, orders, and order items. This raw data is rarely clean, and usually contains missing values, duplicate records, invalid dates, and mismatched identifiers across tables. Before any meaningful analysis can be performed, this data must be cleaned, validated, and organized into a proper relational structure.

This assignment simulates that real-world situation using four related datasets — customers, products, orders, and order_items. The datasets were cleaned using Python and Pandas, loaded into a MySQL database with proper primary and foreign key relationships, and analyzed using SQL. A Python command-line tool was then built on top of SQLite to generate business reports on demand.

3. Implementation Steps

The notebook and scripts were implemented step by step, following a structured data-analytics workflow. The key steps are summarized below.

- **1. Load Datasets** – Loaded the customers, products, orders, and order_items CSV files into Pandas DataFrames.
- **2. Explore Data** – Checked the number of rows, columns, data types, and missing values in each dataset.
- **3. Handle Missing Values** – Identified missing values and applied an appropriate handling strategy for each column.
- **4. Remove Duplicates** – Identified and removed duplicate records to prevent inflated results during analysis.
- **5. Validate Data Types** – Converted dates, quantities, prices, and discount percentages into correct data types.
- **6. Check Referential Integrity** – Validated that customer_id, order_id, and product_id values matched correctly across tables.
- **7. Export Cleaned Data** – Saved the cleaned datasets as customers_clean.csv, products_clean.csv, orders_clean.csv, and order_items_clean.csv.
- **8. Create the Database** – Created the ecommerce_analytics MySQL database with primary and foreign key relationships.
- **9. Load Data into MySQL** – Imported the cleaned CSV files into the database tables using MySQL Workbench.
- **10. Write SQL Analytics** – Implemented joins, aggregations, window functions, and CTEs for revenue, customer, and retention analysis.
- **11. Build the Reporting CLI** – Developed a Python command-line tool (report_cli.py) using SQLite to generate business reports.
- **12. Validate and Test** – Tested the CLI with valid and invalid inputs and saved sample report outputs.

4. Working Flow of the Pipeline

The overall pipeline followed a simple and logical flow, similar to how analytics workflows operate in real data engineering systems. The flow can be summarized as follows.

4.1 Data Cleaning and Validation

The raw CSV datasets were loaded into Pandas DataFrames. Missing values and duplicate records were identified and handled, and important columns such as dates, prices, and quantities were

converted to correct data types. Referential integrity between customers, orders, and order_items was validated before export.

4.2 Relational Database Design

The cleaned datasets were loaded into a MySQL database named `ecommerce_analytics`. Customers, products, orders, and order_items were organized as separate tables connected through primary and foreign key relationships, preserving data integrity across the schema.

4.3 SQL-Based Analytics

SQL queries were written to calculate revenue by category, identify top customers, analyze monthly order trends, and detect product return issues, using joins and aggregation functions on the cleaned relational data.

4.4 Window Functions and CTEs

Window functions such as `SUM() OVER()`, `DENSE_RANK()`, and `LAG()` were used for running totals, product ranking, and purchase-gap analysis. Common Table Expressions were used to structure multi-step analysis such as cohort analysis, customer value quartiles, and year-over-year revenue comparison.

4.5 Command-Line Reporting Tool

A Python CLI was built using the `sqlite3` library to generate revenue, top-customer, monthly-summary, and retention reports based on report type and date-range arguments. The tool also validates inputs and handles edge cases such as invalid dates, empty results, and missing files.

5. Results

The table below summarizes the reports generated by the command-line reporting tool.

Report Type	Output
revenue	Total orders, total revenue, unique customers, revenue change, and revenue by category
top_customers	Top customers ranked by total revenue
monthly_summary	Month-wise orders, customers, and revenue
retention	Active customers, retained customers, and retention percentage

All four report types were successfully generated using sample date ranges, and edge cases such as invalid report names, invalid dates, and empty result sets were handled without the program failing.

6. Advantages Observed

- A single structured pipeline handled data cleaning, storage, and analytics together.
- SQL joins, window functions, and CTEs enabled advanced multi-step analysis without external tools.
- Primary and foreign key relationships kept the relational data consistent and reliable.
- The command-line tool made business reports easily accessible for any given date range.
- Input validation and edge-case handling made the system robust against invalid or missing data.

7. Conclusion

In this assignment, raw e-commerce data across customers, products, orders, and order_items was cleaned and validated using Python and Pandas. The cleaned data was organized into a relational MySQL database, and SQL analytics involving joins, aggregations, window functions, and CTEs were used to derive revenue, customer, and retention insights. A Python command-line tool built on SQLite made these insights accessible through simple report commands.

Overall, this implementation demonstrates a complete and reliable order-analytics workflow, and reflects how real-world data engineering teams turn raw business data into actionable reports.